

STRATÉGIE DE TEST D'ENTREPRISE

Programme Orion — Plateforme ERP des Services à la Personne (Oui Care)

Client / Groupe	Oui Care
Programme	Orion — ERP Unifié SAP
Objet	Gouvernance qualité et stratégie de test
Périmètre	Modules MP1 à MP7, transverses, mobile Hyperion
Version	V1.0
Date	01/07/2026
Statut	Document de gouvernance qualité
Cadre de référence	ISTQB Advanced • Quality Engineering • DevOps
Destinataires	Direction de Programme, PO, BA, Architectes, Dev, QA

CONFIDENTIEL — USAGE INTERNE PROGRAMME ORION

Récapitulatif : Ce document présente la stratégie de test d'entreprise pour le programme Orion, qui est la future plateforme ERP unifiée du groupe Oui Care, couvrant l'ensemble de la chaîne de valeur des services à la personne. Cette stratégie vise à assurer la qualité, la conformité et la performance du produit à travers une démarche intégrée de Quality Engineering (QE) fondée sur cinq piliers : Shift Left, Shift Right, Automation First, Continuous Testing, et Risk Based Testing. Le programme traite un périmètre fonctionnel allant des modules MP1 à MP7, incluant la gestion structurelle, commerciale, planning, facturation, paie, ainsi qu'une application mobile Hyperion. La gouvernance qualité est formalisée et s'appuie sur des indicateurs clés, une cartographie des risques, une pyramide de tests et une automatisation progressive, avec un plan d'action priorisé pour combler les écarts actuels

Sommaire

Sommaire	3
1. Executive Summary	6
1.1 Vision qualité	6
1.2 Enjeux du programme	6
1.3 Objectifs stratégiques.....	6
1.4 Ambition qualité et niveau de maturité visé.....	7
2. Contexte Projet.....	8
2.1 Présentation d'Orion	8
2.2 Architecture fonctionnelle	8
2.3 Architecture technique.....	8
2.4 Modules concernés (MP1 à MP7)	9
2.5 Contraintes du programme.....	10
3. Vision Qualité	11
3.1 Quality Engineering (QE)	11
3.2 Shift Left — tester au plus tôt	11
3.3 Shift Right — observer en production.....	11
3.4 Automation First — automatiser par défaut.....	11
3.5 Continuous Testing — tester en continu.....	11
3.6 Risk Based Testing — prioriser par le risque	12
3.7 Quality Gates — points de contrôle bloquants	12
4. Objectifs Qualité.....	13
4.1 Réduction des défauts.....	13
4.2 Réduction des régressions.....	13
4.3 Accélération des releases	13
4.4 Traçabilité de bout en bout	13
4.5 Couverture automatisée.....	13
4.6 Réduction du coût de la qualité.....	13
4.7 Synthèse des KPI quantifiés.....	14
5. Périmètre de Test	15
5.1 MP1 — Structure & Marques	15
5.2 MP2 — Catalogue Produits (PIM).....	15
5.3 MP3 — Collaborateurs (RH / GTA)	15
5.4 MP4 — Gestion Commerciale (CRM)	16

5.5 MP5 — Planning & Interventions	16
5.6 MP6 — Facturation & Aides Financières	17
5.7 MP7 — Paie	17
6. Cartographie des Risques	18
6.1 Méthode d'évaluation	18
6.2 Matrice des risques par catégorie	18
6.3 Synthèse des risques par module	19
7. Niveaux de Test	20
7.1 Pyramide de test cible	20
7.2 Tests Unitaires	20
7.3 Tests d'Intégration	21
7.4 Tests Contract (API, Kafka, OpenAPI)	21
7.5 Tests API	22
7.6 Tests End-to-End (E2E)	22
7.7 Tests de Régression	23
7.8 Tests Exploratoires	23
7.9 Tests de Performance	23
7.10 Tests de Sécurité	23
7.11 Tests d'Accessibilité	24
7.12 Tests de Compatibilité	24
8. Couverture par Module	25
8.1 Matrice de synthèse (répartition de l'effort)	25
8.2 Objectifs de couverture et d'automatisation par module	25
8.3 Cas prioritaires détaillés (extraits, traçables Jira)	26
9. Tests Case Fonctionnels	26
9.3 Flux inter-modules cibles	27
9.5 Matrice de couverture des chaînes	28
9.6 Jeux de données et gestion des dépendances	28
11.1 Outillage d'automatisation	30
11.2 Critères d'automatisation	31
11.3 Priorisation (score de risque)	31
11.4 Phases d'automatisation	31
11.5 ROI de l'automatisation	32
11.6 Maintenance et gestion des tests instables (flaky)	32
12. Outillage	32

13. Gouvernance	34
13.1 Rôles et responsabilités.....	34
13.2 Matrice RACI des activités de test	34
13.3 Instances et rituels qualité	35
14. Critères d'Entrée (Entry Criteria)	35
15. Critères de Sortie (Exit Criteria)	36
15.1 Critères de Release Readiness (go/no-go).....	36
16. Indicateurs Qualité	37
16.1 Codes couleur des seuils (pilotage visuel)	37
16.2 Alertes automatiques	38
17. Roadmap.....	38
17.1 Phase 1 — Fondations (quick wins).....	38
17.2 Phase 2 — Extension	39
17.3 Phase 3 — Consolidation & GA.....	39
17.4 Trajectoire de couverture par jalon.....	39
18. Conventions et Industrialisation.....	39
18.1 Intégration continue — stages et déclencheurs.....	39
18.2 Stratégie de virtualisation des dépendances externes.....	40
18.3 Conventions de nommage et taxonomie de tags.....	40
18.4 Tests transverses (règles UI communes)	40
18.5 Spécificités mobile (Hyperion, offline-first).....	41
19. Recommandations et Plan d'Action	41
19.1 Écarts constatés vs cible	41
19.2 Plan d'action priorisé.....	42
20. Glossaire et Référence.....	43
20.1 Glossaire	43
20.2 Références documentaires.....	43
20.3 Standards de référence	44
21. Conclusion	44

1. Executive Summary

Orion est le futur ERP stratégique du Groupe Oui Care. Il remplace progressivement un ensemble d'applications historiques et couvre l'intégralité de la chaîne de valeur des services à la personne (SAP) : structure juridique et organisation, habilitations et sécurité, catalogue produits, gestion des collaborateurs, gestion commerciale, planning et interventions, facturation, aides financières, paie, reporting et indicateurs.

La plateforme est distribuée (micro-services Java, front Angular, application mobile Flutter « Hyperion », communication événementielle Kafka, authentification Keycloak multi-tenant, PostgreSQL par domaine, cache Redis). Cette architecture, multi-marque et offline-first, crée une surface de risque qualité élevée : cohérence des flux asynchrones, isolation stricte des données entre marques, calculs financiers sensibles (PEC, paie) et parcours critiques temps réel (planning).

1.1 Vision qualité

La qualité d'Orion n'est pas une phase de fin de cycle : elle est une propriété d'ingénierie continue, intégrée à chaque étape du delivery. Nous adoptons une démarche de Quality Engineering (QE) fondée sur cinq piliers :

- Shift Left (tester au plus tôt),
- Shift Right (observer en production),
- Automation First (automatiser par défaut),
- Continuous Testing (tester en continu dans la CI/CD)
- Risk Based Testing (concentrer l'effort là où le risque est le plus fort).

Énoncé de vision. « Livrer un ERP fiable, conforme et performant, où chaque règle de gestion est couverte par un test exécutable, chaque interface est validée par contrat, et chaque release franchit des quality gates objectives avant d'atteindre la production. »

1.2 Enjeux du programme

- **Continuité opérationnelle.** Orion pilote des interventions à domicile quotidiennes ; un défaut sur le planning (MP5) a un impact direct sur le terrain et sur les bénéficiaires.
- **Exactitude financière.** Les modules Facturation/Aides (MP6) et Paie (MP7) manipulent des calculs réglementaires (PEC, crédit d'impôt, majorations) où toute erreur se traduit en perte financière ou en non-conformité.
- **Isolation multi-marque.** 1 instance Orion = 1 marque ; une fuite de données inter-tenant (cross-tenant leakage) serait un incident de sécurité et RGPD majeur.
- **Conformité réglementaire.** SEGUR santé (INS, DUI, DMP via iCanopée), RGPD, Code du travail imposent des exigences testables et traçables.
- **Coexistence avec l'existant.** Orion s'interface avec le socle historique (Odyssée, ILINK, Pléiade) durant la transition ; la non-régression de ces interfaces est critique.

1.3 Objectifs stratégiques

La stratégie de test poursuit six objectifs stratégiques, déclinés en indicateurs mesurables (section 4) :

#	Objectif stratégique	Ambition cible
O1	Réduire la densité de défauts livrés	< 1 défaut majeur / 1000 lignes de code ; 0 défaut critique en production

#	Objectif stratégique	Ambition cible
02	Maîtriser la non-régression	100 % des parcours critiques automatisés et rejoués à chaque release
03	Accélérer les releases en sécurité	Feedback CI < 15 min (unitaire + intégration) ; go/no-go automatisé
04	Garantir la traçabilité bout-en-bout	Exigence → Règle de gestion → Test → Exécution → Anomalie (Jira + Xray)
05	Industrialiser l'automatisation	≥ 80 % de couverture unitaire ; pyramide de test équilibrée
06	Réduire le coût de la qualité	Diviser par 10 le coût d'une campagne de régression vs manuel

1.4 Ambition qualité et niveau de maturité visé

L'ambition est de faire passer le programme d'une maturité de test réactive (tests manuels tardifs, outillage Xray sous-exploité, aucune stratégie formalisée) à une maturité optimisée (Continuous Testing piloté par le risque, quality gates bloquantes, observabilité production). Le tableau ci-dessous positionne l'état actuel constaté et la cible par jalon.

Axe de maturité	État actuel constaté	Cible V0.1 (02/26)	Cible V1.0 GA (08/26)
Stratégie de test	Absente (aucune page Confluence)	Formalisée et diffusée	Pilotée par KPI, revue par jalon
Automatisation	Amorcée (Playwright E2E, quelques specs)	Socle unitaire + contrat + 5 E2E	Pyramide complète, ROI atteint
Outillage Xray	Présent mais non relié aux US	Traçabilité Test → Story activée	Dashboards consolidés CI → Xray
Quality gates	Inexistantes en CI	Gates G1–G6 bloquantes	Gates G1–G8 (dont perf/sécurité)
Environnements	3 (DEV1, INT1, SQE1)	+ jeux de données maîtrisés	Chaîne iso-prod + pré-prod

Constat structurant issu de l'analyse Jira / Confluence

Aucune stratégie de test formalisée n'existe à ce jour dans l'espace Confluence Orion : le présent document constitue le référentiel de gouvernance qualité du programme.

Backlog volumineux et actif : ~30 Epics, plus de 1000 User Stories, plus de 500 anomalies référencées (projet Jira ORI), avec des modules matures (MP3, MP5) et d'autres non démarrés (MP2, MP7).

Outillage présent mais non gouverné : Xray est installé mais non relié aux US ; aucun composant, version (fixVersion) ni label n'est utilisé dans Jira, ce qui limite le pilotage. Ces manques font l'objet de recommandations explicites dans ce document.

2. Contexte Projet

2.1 Présentation d'Orion

Orion est une plateforme SaaS unifiée de gestion des services à la personne, développée pour Oui Care et ses réseaux (succursales et franchises). Elle vise à remplacer les outils métier fragmentés par une plateforme unique couvrant le cycle complet : de la structuration juridique (marque / société / établissement / agence) jusqu'à la facturation et la paie, en passant par le catalogue produits (PIM), la gestion commerciale, la planification d'interventions à domicile et les aides financières.

La première livraison majeure, Orion V0.1, cible une recette client au premier trimestre 2026 sur un périmètre incluant la structure, les habilitations, le catalogue, le commercial (clients particuliers et professionnels), le planning et les aides (règlothèque). La trajectoire se poursuit vers V0.2 puis la disponibilité générale (GA).

2.2 Architecture fonctionnelle

La chaîne de valeur fonctionnelle s'enchaîne selon le flux suivant, chaque maille alimentant la suivante :

- **Structure (MP1)** → socle organisationnel et habilitations (cône de visibilité, profils, Keycloak).
- **Catalogue / PIM (MP2)** → familles, produits (services / biens / packages), options, frais, TVA.
- **Collaborateurs (MP3)** → intervenants et opérationnels agence, contrats, compétences, disponibilités, GTA.
- **Commercial (MP4)** → prospects et clients PP/Pro, qualification, devis, contrats, parcours SEGUR santé.
- **Planning (MP5)** → vues clients/intervenants, séries récurrentes (RRULE), annulation, pointage, matching, notifications.
- **Facturation & Aides (MP6)** → règlethèque des prises en charge (PEC), organismes financeurs, collecte de facturation.
- **Paie (MP7)** → transformation des heures et absences en variables de paie, exports.

Deux dimensions transverses structurent l'ensemble : les habilitations / sécurité (SP1.5, Keycloak) et le pack réglementaire (jours fériés, vacances scolaires, barèmes), sans oublier les notifications qui font du planning un véritable hub événementiel.

2.3 Architecture technique

La stack technique cible est résumée ci-dessous. Elle guide directement les niveaux et outils de test (section 7).

Composant	Technologie	Enjeu de test associé
Frontend Web	Angular + Angular Material	Test composants, E2E Playwright, accessibilité
Application mobile	Flutter / Dart (projet Hyperion)	Offline-first, SQLite chiffré, sync BFF, tests device
Backend	Micro-services Java (1 domaine = 1 service)	Tests unitaires, intégration Testcontainers, contrat
Authentification / SSO	Keycloak (multi-tenant, realms par marque, JWT)	Isolation tenant, RBAC, tokens, SSO
Communication événementielle	Apache Kafka (événements asynchrones)	Contract testing, idempotence, résilience, DLQ

Composant	Technologie	Enjeu de test associé
Base de données	PostgreSQL (une instance RDS par domaine)	Intégrité, migrations, requêtes, pagination
Cache distribué	Redis (anti thundering-herd, TTL courts)	Invalidation, singleflight, fallback
Push mobile	Firebase Cloud Messaging (FCM)	Notification silencieuse → sync, debounce
Récurrence planning	RRULE RFC 5545 (iCalendar)	Expansion séries, exclusions fériés/vacances
APIs externes	SIRENE (INSEE), GEOWS, iCanopée (SEGUR)	Mocks/virtualisation, cas d'erreur, timeouts
CI/CD	GitHub + GitLab	Pipelines, quality gates, orchestration des suites
Gestion projet / specs / tests	Jira (ORI) + Confluence + Xray	Traçabilité exigence → test → anomalie

2.4 Modules concernés (MP1 à MP7)

Module	Périmètre fonctionnel	Criticité	État backlog Jira (constat)
MP1 — Structure & Marques	Hiérarchie marque/société/agence, habilitations, cône, Keycloak, RGPD	TRÈS HAUTE	Actif — Epics ORI-7/8/9/1354
MP2 — Catalogue (PIM)	Familles, produits, options, frais, TVA, historisation	HAUTE	Non démarré en tant qu'Epic MP2
MP3 — Collaborateurs	Intervenants, contrats, compétences, absences, GTA	HAUTE	Module le plus mature — CdA Gherkin
MP4 — Commercial (CRM)	Clients PP/Pro, devis, contrats, SEGUR, SIRENE	HAUTE	Actif — Epics ORI-64/185
MP5 — Planning	Vues, RRULE, batch, annulation, matching, pointage	CRITIQUE	Actif — Epics ORI-46/107/454/576
MP6 — Facturation & Aides	Règlothèque, OF, PEC, plans d'aide, collecte	CRITIQUE	Actif — Epics ORI-149/182/183
MP7 — Paie	Heures → variables de paie, exports	HAUTE	Sous-documenté — risque de couverture

Note de périmètre. La correspondance $MP \leftrightarrow SP \leftrightarrow Epic$ suit la nomenclature Confluence/Jira « $MP\{n\} - SP\{n\}.\{m\} - E\{n\}.\{m\}.\{k\}$ ». Les habilitations sont portées par SP1.5 (rattachées à MP1). Le match client/intervenant (SP5.1bis) est prévu en V0.3.

2.5 Contraintes du programme

Nature	Contrainte	Conséquence sur la stratégie de test
Calendrier	Recette V0.1 T1 2026, puis V0.2 (05/26) et GA (08/26)	Priorisation par le risque, automatisation incrémentale par jalon
Environnements	3 environnements seulement (DEV1, INT1, SQE1) ; pas de PREPROD/PROD documentés	Discipline sur les jeux de données ; recommandation d'ajouter PREPROD
Données	Multi-tenant, RGPD, données de santé (SEGUR)	Anonymisation, préfixes TEST_AUTO_, isolation par agency_id
Dépendances externes	SIRENE, GEOWS, iCanopée non maîtrisables	Virtualisation de service (mocks) obligatoire
Outillage	Xray non relié aux US ; pas de composants/versions Jira	Mise en place de la traçabilité et du référentiel de test
Organisation	Équipes par module + besoin d'une QA transverse	RACI clair, ownership des tests par équipe, coordination inter-modules

3. Vision Qualité

Cette section définit le cadre conceptuel de la démarche qualité d'Orion. Chaque concept est traduit en pratiques concrètes et en implications pour le programme, afin qu'il ne reste pas théorique mais devienne opérationnel.

3.1 Quality Engineering (QE)

Le Quality Engineering déplace la responsabilité de la qualité : elle n'appartient plus à une équipe de test en fin de chaîne, mais à l'ensemble des acteurs du delivery, tout au long du cycle. Le rôle de la QA évolue d'une fonction de contrôle à une fonction d'ingénierie, d'outillage et d'animation de la qualité. Concrètement : les développeurs possèdent leurs tests unitaires et d'intégration ; la QA transverse possède l'automatisation E2E, les tests non fonctionnels et le pilotage des indicateurs ; l'ensemble partage un référentiel unique (Jira / Xray).

3.2 Shift Left — tester au plus tôt

- **Définition.** Anticiper les activités de test dès l'expression du besoin et du code, plutôt que de les concentrer en aval.
- **Pratiques Orion.** Revue de testabilité des User Stories (chaque US porte des critères d'acceptation exploitables) ; règles de gestion (RG_*) couvertes par au moins un test unitaire ; TDD sur les moteurs de calcul sensibles (PEC, paie, RRULE) ; contract-first sur les APIs et les messages Kafka.
- **Gate d'entrée.** Une US n'entre en développement que si elle satisfait la Definition of Ready (DoR) définie sur Confluence.

3.3 Shift Right — observer en production

- **Définition.** Prolonger la qualité au-delà de la mise en production par l'observabilité, la supervision et les tests en conditions réelles.
- **Pratiques Orion.** Supervision des métriques applicatives et d'infrastructure (Prometheus / Grafana) ; surveillance des files Kafka et des DLQ (dead-letter queues) ; smoke tests post-déploiement ; suivi des taux d'erreur des APIs externes ; boucle de retour des anomalies production vers le backlog de test (analyse de defect leakage).

3.4 Automation First — automatiser par défaut

- **Définition.** L'automatisation est le choix par défaut ; le test manuel est réservé à l'exploratoire, à l'ergonomie et aux cas à faible ROI d'automatisation.
- **Pratiques Orion.** Tout parcours critique rejoué plus d'une fois est candidat à l'automatisation ; les critères d'automatisation, la priorisation et le ROI sont formalisés (section 11) ; l'automatisation s'appuie sur Playwright (web), le socle unitaire/intégration (back) et Pact (contrat).

3.5 Continuous Testing — tester en continu

Les tests sont intégrés au pipeline CI/CD et s'exécutent automatiquement à chaque changement, avec des boucles de feedback courtes. La stratégie définit quelle suite s'exécute à quel déclencheur (push, merge request, nightly, pré-release), avec des budgets de temps stricts (section 7 et section 11).

Déclencheur	Suites exécutées	Budget de temps cible
Push (branche feature)	Build + Tests unitaires	< 5 min

Déclencheur	Suites exécutées	Budget de temps cible
Merge Request	Build + Unitaire + Intégration + Contrat	< 15 min
Merge sur develop	+ E2E smoke (parcours critiques)	< 20 min
Nightly	Suite complète (E2E, perf smoke, sécurité scan)	< 60 min
Pré-release (release/*)	Complet + E2E étendu + perf + sécurité	< 45 min (hors perf full)

3.6 Risk Based Testing — prioriser par le risque

L'effort de test est alloué proportionnellement au risque, évalué comme la combinaison de la probabilité d'occurrence d'un défaut et de son impact. Cette approche gouverne la profondeur de test par module (section 5), la cartographie des risques (section 6) et la priorisation de l'automatisation (section 11). Les modules CRITIQUES (MP5 Planning, MP6 Facturation) concentrent l'effort le plus intense.

3.7 Quality Gates — points de contrôle bloquants

Les quality gates sont des critères objectifs et automatisés, alignés sur la Definition of Done (DoD), qu'un incrément doit franchir pour progresser dans le pipeline. Une gate non franchie bloque la progression. Elles matérialisent l'engagement qualité du programme.

Gate	Critère (aligné DoD)	Vérification	Bloquant
G1 — Build	Compilation sans erreur	Étape build CI	Oui
G2 — Unitaire	Tests unitaires verts, couverture ≥ 75 %	JUnit / Jasmine + JaCoCo / Istanbul	Oui
G3 — Intégration	Tests d'intégration verts, couverture ≥ 60 %	Test containers + REST Assured	Oui
G4 — Code / SonarQube	0 Vulnérabilité critique, 0 bug bloquant	Quality Gate SonarQube	Oui
G5 — Contrat	Contrats API et Kafka valides	Pact / can-i-deploy	Oui
G6 — Revue	Code revu et approuvé par ≥ 1 pair	Approbation Merge Request	Oui
G7 — E2E	Parcours critiques verts	Playwright	Oui (release)
G8 — Sécurité	Scan OWASP sans alerte haute	OWASP ZAP + Dependency-Check	Oui (release)

4. Objectifs Qualité

Les objectifs qualité déclinent les objectifs stratégiques (section 1.3) en cibles quantifiées et mesurables. Chaque objectif est associé à un ou plusieurs indicateurs suivis dans le temps (le détail complet des KPI de gouvernance figure en section 16).

4.1 Réduction des défauts

- **Cible.** Densité de défauts < 1 défaut majeur pour 1000 lignes de code ; zéro défaut critique atteignant la production.
- **Levier.** Couverture unitaire élevée, revue de code systématique, gate SonarQube, tests des règles de gestion.

4.2 Réduction des régressions

- **Cible.** 100 % des parcours critiques automatisés et rejoués à chaque release ; taux de détection des régressions avant production > 95 %.
- **Levier.** Suite de non-régression automatisée (Playwright + contrat), exécution en continu, tags @regression et @critical.

4.3 Accélération des releases

- **Cible.** Feedback CI unitaire + intégration < 15 min ; décision go/no-go de release automatisée par les quality gates.
- **Levier.** Pyramide de test équilibrée (tests rapides majoritaires), parallélisation, virtualisation des dépendances.

4.4 Traçabilité de bout en bout

La traçabilité relie chaque exigence à sa validation : Exigence / User Story (Jira) → Règle de gestion (Confluence, RG_*) → Cas de test (Xray) → Exécution (Xray / CI) → Anomalie (Jira). Cible : 100 % des User Stories critiques couvertes par au moins un test tracé dans Xray.

4.5 Couverture automatisée

- **Cible unitaire.** ≥ 80 % (seuil bloquant 75 %).
- **Cible intégration.** ≥ 70 % (seuil bloquant 60 %).
- **Cible contrat.** 100 % des interfaces Kafka et API (OpenAPI).
- **Cible E2E.** 100 % des parcours critiques (web et mobile).

4.6 Réduction du coût de la qualité

L'automatisation vise à réduire drastiquement le coût et le délai des campagnes de non-régression par rapport à l'approche manuelle actuelle. Le tableau ci-dessous illustre le gain attendu (ordres de grandeur, à affiner avec les coûts réels du programme).

Critère	Approche manuelle (actuelle)	Approche automatisée (cible)
Durée d'une campagne de régression	~ 5 jours-personne	~ 50 min (exécution CI)

Critère	Approche manuelle (actuelle)	Approche automatisée (cible)
Coût par exécution	élevé (mobilisation d'équipe)	marginal (infrastructure CI)
Fréquence réaliste	1 fois par release	à chaque push / merge
Couverture de régression	~ 40 % des parcours	100 % des parcours critiques
Délai de détection d'une régression	J+5 à J+10	< 30 min

4.7 Synthèse des KPI quantifiés

Vue d'ensemble des indicateurs cibles ; le pilotage détaillé (seuils d'alerte, fréquence, responsable) est en section 16.

Indicateur	Objectif	Seuil d'alerte	Fréquence
Couverture tests unitaires / module	≥ 80 %	< 75 %	Chaque build
Couverture tests intégration / module	≥ 70 %	< 60 %	Chaque build
Taux de réussite (pass rate)	≥ 98 %	< 95 %	Chaque build
Taux de tests instables (flaky)	< 2 %	> 5 %	Hebdomadaire
MTTR d'un test cassé	< 4 h	> 8 h	Continu
Détection de régression pré-prod	> 95 %	< 90 %	Par release
Défauts critiques en production	0	≥ 1	Par sprint / release
Temps d'exécution suite complète	< 50 min	> 75 min	Quotidien
Vulnérabilités critiques ouvertes	0	≥ 1	Par scan

5. Périmètre de Test

Cette section découpe le périmètre de test par module (MP1 à MP7). Pour chacun sont identifiés les fonctionnalités clés, les flux métier, les interfaces et les risques principaux. La profondeur de test par niveau et les cibles de couverture/automatisation sont détaillées en section 8 (couverture par module).

5.1 MP1 — Structure & Marques

Criticité : **TRÈS HAUTE**

Dimension	Description
Fonctionnalités clés	Hiérarchie marque / société / établissement / agence (CRUD) ; profils et droits ; cône de visibilité ; synchronisation Keycloak ; import/export annuaire ; RGPD.
Flux métier	Alimente tous les modules (rattachement). Crée les comptes utilisateurs via Kafka vers Keycloak. Définit le cône de visibilité appliqué en MP4/MP5.
Interfaces	Keycloak (realms, rôles, SSO) ; Kafka (init comptes/groupes) ; import CSV.
Risques principaux	Cône mal résolu → fuite de données inter-périmètre (critique) ; désynchronisation Keycloak↔Orion ; import CSV corrompu ; performance sur arbres profonds.

5.2 MP2 — Catalogue Produits (PIM)

Criticité : **HAUTE**

Dimension	Description
Fonctionnalités clés	Familles et catégories ; produits (services, biens, packages) ; options et frais ; TVA ; cycle de vie (brouillon → actif → désactivé) ; historisation des versions.
Flux métier	Alimente les devis (MP4) et l'affichage des familles au planning (MP5). Fournira à terme le délai de prévenance d'annulation (CGV).
Interfaces	Contrat API SIRENE (fiabilisation client Pro) ; consommation par MP4 et MP5.
Risques principaux	Modification tarifaire impactant à tort des devis existants (rupture de snapshot) ; incohérence d'un package si un composant est désactivé ; perte d'historique.

5.3 MP3 — Collaborateurs (RH / GTA)

Criticité : **HAUTE**

Dimension	Description
Fonctionnalités clés	Intervenants et opérationnels agence ; contrats ; compétences ; disponibilités et indisponibilités ; absences et congés ; gestion des temps (GTA), annualisation ATT ; suivi RH.
Flux métier	Alimente le planning (MP5) via événements Kafka (collaborateur, conge, autre_absence) et la paie (MP7). Crée automatiquement le compte utilisateur à la signature du contrat.

Dimension	Description
Interfaces	Kafka MP3→MP5 (I1, congés/absences) ; Keycloak (création compte) ; interfaces historiques ILink / Pléiade ; import Digital Recruiters.
Risques principaux	Message Kafka perdu → intervenant invisible au planning (critique) ; calcul d'annualisation erroné ; exposition de données RH hors cône.

5.4 MP4 — Gestion Commerciale (CRM)

Criticité : **HAUTE**

Dimension	Description
Fonctionnalités clés	Prospects et clients PP/Pro ; qualification du besoin ; devis ; contrats ; multi-sites (mère/filles) ; parcours SEGUR santé (INS, DUI, consentement, DMP/MSSanté).
Flux métier	Déclenche le planning (MP5) et la facturation (MP6). Émet des événements Kafka (client, client_pro, J1/J2). Publie les absences client vers MP5.
Interfaces	SIRENE (enrichissement Pro) ; iCanopée (INS, DMP, MSSanté) ; Pro Santé Connect (step-up Keycloak + CIBA) ; Kafka MP4→MP5.
Risques principaux	Enrichissement SIRENE erroné / doublons ; gestion du cycle INS (Provisoire→Récupérée→Validée→Qualifiée) ; verrouillage des traits stricts ; consentement e-santé non respecté.

5.5 MP5 — Planning & Interventions

Criticité: **CRITIQUE**

Dimension	Description
Fonctionnalités clés	Vues planning (clients / intervenants) ; CRUD interventions ; séries récurrentes (RRULE) ; batch de génération « au fil de l'eau » ; annulation (motif x délai) ; pointage ; matching client/intervenant ; hub de notifications.
Flux métier	Cœur opérationnel. Alimenté par MP1/MP2/MP3/MP4. Alimente MP6 (interventions réalisées/pointées) et MP7 (heures). Consomme les absences client (Kafka MP4→MP5).
Interfaces	Kafka (consommation absences, émission D4/D5/D6..., I1/J1/J2/K1/K2/H2) ; RRULE ; pack réglementaire (fériés/vacances).
Risques principaux	Batch RRULE générant des interventions erronées (critique) ; facturabilité mal calculée à l'annulation (perte financière) ; performance de la vue semaine ; race conditions sur modifications simultanées ; message absence non-traité → DLQ.

5.6 MP6 — Facturation & Aides Financières

Criticité : **CRITIQUE**

Dimension	Description
Fonctionnalités clés	Règlothèque des prises en charge (PEC) ; organismes financeurs (OF) ; conventions (CDA) ; plans d'aide personnalisés (PAP) ; éligibilité ; collecte de facturation ; crédit d'impôt. En V0.1: alimentation par import Excel (pas d'IHM complète).
Flux métier	Consomme les interventions réalisées de MP5 (table de faits). Utilise les contrats de MP4 pour le calcul PEC. Exporte vers la comptabilité.
Interfaces	Kafka MP5→MP6 ; import Excel ; moteur de règles DMN ; interface avec MP7 (table partagée brouillon_facturation) ; URSSAF (avance immédiate / crédit d'impôt).
Risques principaux	Erreur de calcul PEC → facturation incorrecte (critique) ; import Excel silencieusement erroné ; cumul multi-aides dépassant 100 % ; changement réglementaire non répercuté.

5.7 MP7 — Paie

Criticité: **HAUTE**

Dimension	Description
Fonctionnalités clés	Transformation des heures (MP5) et absences (MP3) en variables de paie ; heures supplémentaires et majorations ; export vers le système de paie (Pléiade) ; réconciliation. Module encore peu spécifié à ce stade.
Flux métier	Consomme les heures de MP5 et les absences de MP3. Interface avec MP6 via table partagée. Exporte les variables de paie.
Interfaces	Table partagée brouillon_facturation (MP6) ; export Pléiade ; GTA (MP3).
Risques principaux	Calcul de majorations erroné (conformité légale) ; export non conforme ; écart heures MP5 vs pointages GTA ; spécifications incomplètes → tests à définir.

NB : Ce module n'est pas encore spécifié (à Revoir)

6. Cartographie des Risques

La cartographie des risques est le moteur du Risk Based Testing. Chaque risque est évalué selon sa probabilité d'occurrence et son impact, dont le produit détermine la criticité. La criticité oriente la profondeur de test, la priorité d'automatisation et l'ordre de traitement.

6.1 Méthode d'évaluation

La probabilité et l'impact sont notés sur une échelle de 1 (faible) à 5 (très élevé). La criticité est le produit des deux (échelle 1 à 25).

Criticité (P × I)	Niveau	Politique de test associée
15 – 25	Critique	Couverture maximale, automatisation prioritaire, tests exhaustifs, revue dédiée
8 – 14	Élevé	Couverture forte, automatisation des cas nominaux + erreurs principales
4 – 7	Modéré	Couverture ciblée sur les cas nominaux, tests exploratoires
1 – 3	Faible	Couverture minimale, surveillance

6.2 Matrice des risques par catégorie

Les risques majeurs du programme, classés par catégorie, avec leur évaluation et leur mitigation.

Catégorie	Risque	P	I	Crit.	Niveau	Mitigation
Fonctionnel	Batch RRULE génère des interventions erronées (dates, exclusions fériés)	4	5	20	Critique	Property-based + golden tests, dry-run, exclusions pack réglementaire
Fonctionnel	Facturation mal calculée à l'annulation (motif × délai)	3	5	15	Critique	Tests exhaustifs de la matrice motif × délai (RG_GIN_DPG_*)
Données	Erreur de calcul PEC → facturation incorrecte	3	5	15	Critique	Barèmes réels, double calcul shadow, revue métier
Sécurité	Fuite de données inter-tenant (cross-tenant leakage)	2	5	10	Élevé	Isolation realms Keycloak, tests systématiques, pen-testing
Sécurité	Contournement du cône de visibilité / IDOR	3	5	15	Critique	Property-based testing habilitations, 403/404 sur bypass API
Interfaces	Message Kafka perdu → entité invisible (planning)	3	5	15	Critique	Idempotence + retry, DLQ, tests de résilience (chaos)
Interfaces	Dépendance externe indisponible (SIRENE, iCanopée)	3	3	9	Élevé	Virtualisation (WireMock), gestion 404/429/503/timeout
Technique	Désynchronisation Keycloak ↔ Orion	3	4	12	Élevé	Health-check + réconciliation automatisée
Performance	Vue planning semaine > SLA (charge)	3	4	12	Élevé	Tests de charge k6 (300+ lignes), budget perf strict CI

Catégorie	Risque	P	I	Crit.	Niveau	Mitigation
Performance	Batch mensuel RRULE trop long à l'échelle	2	4	8	Élevé	Endurance et scalabilité (1000 séries), profilage
Batch	Import Excel (MP6) silencieusement erroné	3	5	15	Critique	Validation exhaustive par ligne, rollback atomique, fuzzing
Mobile	Corruption des données offline / perte à la sync	3	4	12	Élevé	Tests offline-first, intégrité post-sync, kill/restart
Mobile	Server_ts basé sur l'horloge locale (au lieu du delta)	3	4	12	Élevé	Assertion max(updated_at), jamais DateTime.now ()
Multi-tenant	Fuite de configuration ou de cache entre marques	2	5	10	Élevé	Clé de cache par (entity, agency), invalidation ciblée
Données	Non-conformité RGPD (anonymisation, consentement)	2	5	10	Élevé	Anonymisation irréversible, suppression cascade, tests dédiés

Lecture. *P = probabilité (1-5), I = impact (1-5), Crit. = criticité (P×I). Les lignes rouges (criticité ≥ 15) concentrent l'effort de test prioritaire et l'automatisation en première vague.*

6.3 Synthèse des risques par module

Module	Nb risques critiques	Catégories dominantes	Priorité de traitement
MP5 — Planning	3	Fonctionnel, Interfaces, Performance	P0 — immédiate
MP1 — Structure	2	Sécurité (cône, tenant)	P0 — immédiate
MP6 — Facturation	2	Données, Batch	P1 — haute
MP4 — Commercial	1	Interfaces, Fonctionnel	P1 — haute
MP3 — Collaborateurs	1	Interfaces (Kafka)	P2 — moyenne
Hyperion (mobile)	0 (2 élevés)	Mobile, Données	P2 — moyenne
MP7 — Paie	0 (specs à venir)	Données, conformité	P3 — conditionnée aux specs
MP2 — Catalogue	0	Fonctionnel	P3 — selon avancement

7. Niveaux de Test

Cette section décrit chaque niveau de test selon la structure ISTQB : objectifs, responsables, outils et critères. L'ensemble s'organise en pyramide de test, garantissant un socle large de tests rapides et fiables et un sommet réduit de tests bout-en-bout coûteux.

7.1 Pyramide de test cible

Niveau	% Du total	Nb tests estimé	Temps d'exécution	Fréquence	Coût de maintenance
Unitaire	60 %	~ 3 000	< 5 min	Chaque push	Faible
Intégration	20 %	~ 800	< 15 min	Chaque push	Moyen
Contrat	10 %	~ 200	< 5 min	Push affectant une API	Faible
E2E Web	3 %	~ 80	< 30 min	MR + nightly	Élevé
E2E Mobile	1 %	~ 30	< 20 min	Nightly	Élevé
Performance	3 %	~ 40	< 20 min	Nightly + pré-release	Moyen
Sécurité	3 %	~ 30	< 15 min	Nightly + pré-release	Faible

7.2 Tests Unitaires

Aspect	Détail
Objectifs	Valider en isolation la logique métier, les règles de gestion (RG_*), les calculs et les transformations. Chaque RG doit disposer d'au moins un test unitaire dédié.
Responsable	Développeurs de chaque équipe module (ownership complet).
Outils	Back Java: JUnit 5 + Mockito + AssertJ (couverture JaCoCo). Front Angular: Jasmine/Karma (Istanbul). Mobile Flutter: flutter_test + mocktail (lcov).
Périmètre prioritaire	Règles de gestion, logique RRULE (parsing, expansion, exceptions), validations (SIREN/SIRET, NAF, horaires), calculs (PEC, tarification, prorata, majorations paie), mappers DTO↔Entity et transformations Kafka.
Critères (Entrée)	Code compilé ; spécification / règle de gestion disponible.
Critères (Sortie)	100 % des tests verts; couverture ≥ 80 % (seuil bloquant 75 %); convention should_<attendu>_when_<condition>.

7.3 Tests d'Intégration

Les tests d'intégration valident les interactions entre composants et avec l'infrastructure réelle, en s'appuyant sur Test containers pour une isolation complète (conteneurs Docker éphémères par exécution).

Aspect	Détail
Objectifs	Valider la persistance, les APIs REST, la production/consommation Kafka, l'authentification Keycloak et le cache Redis, dans des conditions proches du réel.
Périmètre	Flux critiques : requêtes et pagination PostgreSQL ; endpoints REST (init formulaire, CRUD) ; publication/consommation d'événements ; tokens et isolation de tenant ; invalidation de cache.
Flux Kafka	Production et consommation par entité (collaborateur, conge, client, absence...) ; idempotence ; gestion des messages malformés (routage DLQ).
APIs	REST Assured pour l'assertion des contrats d'API internes ; virtualisation des APIs externes (WireMock).
Base de données	Testcontainers PostgreSQL ; vérification des migrations, contraintes, transactions et rollback.
Outils	Test containers (PostgreSQL, Kafka, Redis, Keycloak) + REST Assured.
Critères (Sortie)	Tests verts ; couverture ≥ 70 % (seuil bloquant 60 %) ; conteneurs démarrés et sains.

Conteneurs Testcontainers de référence

Service	Image Docker	Usage en test
PostgreSQL	postgres:16-alpine	Persistance, requêtes, pagination
Kafka	confluentinc/cp-kafka:7.6	Messaging inter-services
Redis	redis:7-alpine	Cache, anti-thundering-herd
Keycloak	keycloak:24.0	Auth, tokens, realms multi-tenant

7.4 Tests Contract (API, Kafka, OpenAPI)

Le contract testing garantit la compatibilité entre producteurs et consommateurs d'interfaces sans avoir à déployer tous les services simultanément. L'approche retenue est Consumer-Driven Contracts (Pact) avec un broker centralisé et une gate can-i-deploy.

- **API REST.** Chaque consommateur déclare ses attentes ; chaque producteur vérifie qu'il les respecte. Le format d'erreur standardisé (ODR-2) est validé.
- **Kafka.** Le format d'événement normalisé (agency_id, entity_type, entity_id, op, updated_at) est un contrat pivot ; les schémas des messages sont versionnés et vérifiés.
- **OpenAPI.** Les spécifications OpenAPI servent de source de vérité pour la génération et la validation ; toute rupture (breaking change) déclenche une migration coordonnée en deux phases.

Flux	Producteur	Consommateur	Types d'entités
MP3 → MP5	MP3 (RH)	MP5 (Planning)	collaborateur, conge, autre_absence

Flux	Producteur	Consommateur	Types d'entités
MP4 → MP5	MP4 (Commercial)	MP5 (Planning)	client, client_pro, J1, J2, absence client
MP5 → MP6	MP5 (Planning)	MP6 (Facturation)	intervention_realisee / pointage
MP1 → Keycloak	MP1 / SP1.5	Keycloak	init compte / groupe utilisateur
Notifier → BFF	Notifier	BFF Mobile	sync_changelog
BFF → Mobile	BFF	App Flutter (Hyperion)	delta de synchronisation (JSON)

7.5 Tests API

Au-delà du contrat, les tests d'API valident le comportement fonctionnel des endpoints REST : codes de statut, schémas de réponse, pagination, gestion d'erreurs, sécurité (401/403). Ils constituent un niveau intermédiaire rapide et stable, préféré aux tests E2E lorsque la validation ne nécessite pas l'interface. Aucune API GraphQL n'est identifiée à ce stade sur Orion ; l'architecture est REST + événementiel Kafka. Ce point sera réévalué si GraphQL est introduit.

7.6 Tests End-to-End (E2E)

Les tests E2E valident des parcours métier complets à travers l'interface, dans un environnement intégré. Ils sont peu nombreux mais à haute valeur, ciblés sur les parcours critiques. Le détail des chaînes transverses figure en section 9.

- **Web (Angular).** Playwright (TypeScript). Parcours critiques onboarding client, planification, annulation, aides. Stratégie de données : seeding via API, cleanup systématique, préfixe TEST_AUTO_.
- **Mobile (Hyperion / Flutter).** Patrol + integration_test, exécution sur device (Firebase Test Lab) en nightly. Focus offline first, pointage, synchronisation.

#	Parcours critique web	Module
1	Configurer la structure marque / société / agence	MP1
2	Gérer les habilitations (profils, utilisateurs, droits)	SP1.5
3	Créer une famille + produit + options	MP2
4	Onboarder un client particulier (PP)	MP4
5	Onboarder un client professionnel (SIRENE)	MP4
6	Planifier la semaine (vue planning)	MP5
7	Créer une série récurrente (RRULE)	MP5
8	Modifier / annuler une intervention	MP5
9	Matcher client / intervenant	MP5
10	Traiter les notifications	MP5
11	Générer les interventions au fil de l'eau (batch)	MP5
12	Configurer les aides PEC (OF / CDA / PAP)	MP6

#	Parcours critique web	Module
13	Gérer l'identité SEGUR (INS / DUI)	MP4
14	Collecter la facturation + export	MP6
15	Gérer un collaborateur complet	MP3

7.7 Tests de Régression

La régression est assurée par l'ensemble des tests automatisés rejoués à chaque changement. La suite de non-régression est constituée des tests tagués @regression et @critical, exécutée en continu. Objectif : détecter > 95 % des régressions avant la production. La sélection intelligente (impacted tests) peut être introduite pour réduire les temps.

7.8 Tests Exploratoires

Les tests exploratoires complètent l'automatisation par une investigation humaine, non scriptée, guidée par des chartes de test (session-based testing). Ils sont particulièrement adaptés aux nouvelles fonctionnalités, aux parcours complexes (SEGUR, aides) et à la détection de défauts inattendus. Chaque session produit des notes et alimente le backlog d'anomalies et de nouveaux cas automatisables.

7.9 Tests de Performance

Outil : k6 (métriques exportées vers InfluxDB / Grafana). Quatre types complémentaires :

- **Charge (load).** Vérifier le respect des SLA sous la charge nominale attendue.
- **Stress.** Déterminer le point de rupture au-delà de la charge nominale.
- **Endurance (soak).** Détecter les fuites de ressources sur une durée prolongée.
- **Scalabilité.** Mesurer l'évolution des performances avec la volumétrie et la concurrence.

Scénario	SLA cible (p95)	Volume / conditions
Vue planning semaine	< 2 s	500 intervenants × 7 jours
Batch RRULE mensuel	< 60 s	1 000 séries récurrentes
Synchronisation BFF (delta)	< 500 ms	200 entités modifiées
Synchronisation BFF (full load)	< 10 s	10 000 entités (lots de 200)
Recherche collaborateurs	< 1 s	5 000 collaborateurs filtrés
Création d'intervention	< 500 ms	50 utilisateurs concurrents

7.10 Tests de Sécurité

Outils : OWASP ZAP (scan dynamique), SonarQube (analyse statique), Dependency-Check (dépendances). Axes prioritaires :

Catégorie	Objet du test
Multi-tenant	Isolation stricte des realms Keycloak, zéro fuite de données inter-tenant

Catégorie	Objet du test
IAM / Habilitations	Contournement du cône → 403 ; IDOR → 403/404 ; élévation de privilège
SSO / JWT	Token expiré → 401 ; token modifié → rejet de signature ; refresh
OWASP Top 10	Injection SQL sur filtres, XSS sur saisies, mauvaise configuration
Rate limiting	Brute force login → verrouillage ; protection contre l'énumération
RGPD	Anonymisation irréversible, suppression en cascade, respect du consentement

7.11 Tests d'Accessibilité

L'accessibilité (cible RGAA / WCAG 2.1 niveau AA) est vérifiée sur le front Angular par des outils automatisés (axe-core intégré à Playwright, Lighthouse) et des contrôles manuels (navigation clavier, lecteurs d'écran, contrastes). Elle constitue une exigence de qualité produit et, selon le contexte, une obligation réglementaire.

7.12 Tests de Compatibilité

- **Web.** Navigateurs cibles (Chrome, Edge, Firefox, Safari) et résolutions ; exécution multi-navigateurs via Playwright (projets).
- **Mobile.** Matrice d'appareils et versions OS (Android / iOS) via Firebase Test Lab ; gestion des tailles d'écran et des conditions réseau.

8. Couverture par Module

Pour chaque module sont précisés les objectifs de validation, les risques couverts, les cas prioritaires, la couverture cible et le niveau d'automatisation visé. La matrice de synthèse donne la répartition indicative de l'effort par niveau de test.

8.1 Matrice de synthèse (répartition de l'effort)

Module	Unit.	Intég.	API	E2E	Perf	Sécu	Criticité
MP1 Structure	40 %	25 %	20 %	10 %	—	5 %	TRÈS HAUTE
MP2 Catalogue	35 %	30 %	25 %	10 %	—	—	HAUTE
MP3 Collaborateurs	30 %	30 %	20 %	10 %	5 %	5 %	HAUTE
MP4 Commercial	25 %	30 %	25 %	10 %	—	10 %	HAUTE
MP5 Planning	30 %	25 %	20 %	15 %	10 %	—	CRITIQUE
MP6 Facturation & Aides	35 %	30 %	20 %	5 %	5 %	5 %	CRITIQUE
MP7 Paie	40 %	30 %	20 %	—	—	10 %	HAUTE

Lecture. Les pourcentages expriment la répartition relative de l'effort de test au sein du module (et non une couverture absolue). Ils traduisent le profil de risque de chaque module.

8.2 Objectifs de couverture et d'automatisation par module

Module	Couverture cible (unit./intég.)	Automatisation cible	Cas prioritaires
MP1 Structure	85 % / 70 %	Élevée (habilitations, cône)	Hiérarchie CRUD, cône, profils, sync Keycloak, import CSV
MP2 Catalogue	80 % / 70 %	Moyenne	Cycle de vie produit, package, historisation, snapshot devis
MP3 Collaborateurs	80 % / 70 %	Élevée (Kafka, GTA)	Contrat→compte, Kafka I1, absences, annualisation, pointage
MP4 Commercial	80 % / 70 %	Moyenne–Élevée	Onboarding PP/Pro, SIRENE, cycle INS, consentement, DMP
MP5 Planning	85 % / 75 %	Très élevée	Batch RRULE, annulation motif×délai, absence Kafka, charge, matching
MP6 Facturation & Aides	85 % / 70 %	Élevée (calculs)	Chaîne OF→CDA→PAP→PEC, import Excel, cumul aides, collecte
MP7 Paie	85 % / 70 %	TDD (conditionné specs)	Calcul mensuel, heures sup, export Pléiade, réconciliation

8.3 Cas prioritaires détaillés (extraits, traçables Jira)

Sélection de scénarios critiques par module, référencés aux Epics Jira réelles pour la traçabilité.

ID	Scénario critique	Module / Epic
SC-MP1-01	Création hiérarchie complète et vérification de la cascade	MP1 / ORI-8
SC-MP1-03	Cône de visibilité multi-agences : filtrage des données MP4/MP5	MP1 / ORI-9
SC-MP1-04	Synchronisation Keycloak : utilisateur → realm + rôles + SSO	SP1.5 / ORI-9
SC-MP3-02	Kafka collaborateur : message I1 consommé par MP5 → visible planning	MP3 / ORI-1
SC-MP3-04	Annualisation ATT sur 12 mois → alertes de dépassement	MP3 / ORI-2
SC-MP4-02	Client Pro SIRENE : enrichissement auto → site → contrat	MP4 / ORI-185
SC-MP4-03	SEGUR INS : Provisoire → Récupérée → Validée → Qualifiée	MP4 / ORI-64
SC-MP5-01	Batch RRULE : 100 séries → 12 mois, exclusion fériés (RG_BTCH_SIN_*)	MP5 / ORI-46
SC-MP5-03	Annulation : chaque motif × délai → état de facturation correct (RG_GIN_DPG_*)	MP5 / ORI-46
SC-MP5-04	Kafka absence client: valide → UPSERT ; invalide → DLQ	MP5 / ORI-435
SC-MP6-01	Chaîne OF → CDA (APA) → PAP → éligibilité → calcul PEC	MP6 / ORI-149
SC-MP6-02	Import Excel : conforme → création ; malformé → erreurs par ligne	MP6 / ORI-182
SC-MP7-02	Heures sup : majorations 25 % (36-43h) et 50 % (>43h)	MP7 (specs à venir)

9. Tests Case Fonctionnels

9.1 Tests E2E Par-Module

MP1 - Structure (Marque / Societe / Etablissement / Agence)

SP1.5 - Habilitations (Profils / Utilisateurs / Droits)

MP2 - Catalogue (Familles / Produits / Options / Frais)

MP3 - Collaborateurs (Intervenants / Contrats / GTA / Absences)

MP4 - Commercial + SEGUR (Clients / Devis / RDV / Santé)

MP5 - Planning & Interventions

MP6 - Facturation & Aides Financières

MP7 - Paie (GTA / Interface Pléiade)

9.2 Tests E2E Cross-Module

Les tests cross-module valident les flux inter-domaines, là où se concentrent les risques d'intégration les plus élevés. Ils suivent la chaîne de valeur d'Orion et s'appuient sur les flux Kafka documentés. Trois chaînes E2E de référence couvrent l'essentiel du système.

9.3 Flux inter-modules cibles

Flux	Nature	Dépendance / donnée pivot	Cas critique à valider
MP1 → MP3	Rattachement	Agence / société → collaborateur	Un intervenant est rattaché à une agence du cône ; compte créé via Keycloak
MP1 → MP5	Notification (K1/K2)	Agence/société → planning	Création/modif agence propagée au planning
MP3 → MP5	Kafka (I1, congés)	collaborateur, conge, absence	Intervenant + dispos visibles au planning ; congé → désaffectation
MP2 → MP4	Référence produit	produit → devis	Un produit du catalogue alimente un devis
MP4 → MP5	Kafka (J1/J2, absence)	client, contrat, absence client	Client + contrat visibles; absence client → UPSERT planning
MP5 → MP6	Table de faits	intervention réalisée / pointage	Interventions pointées → collecte facturation → PEC appliqué
MP6 → MP7	Table partagée	brouillon_facturation / heures	Heures et variables cohérentes entre facturation et paie

9.4 Chaînes des test E2E (C1 à C9)

9.4.1 Chaîne 1 (C1) — Client particulier à domicile

9.4.2 Chaîne 2 (C2) — Client professionnel multi-sites (nettoyage bureaux)

9.4.3 Chaîne 3 (C3) — Client santé SAAD (SEGUR)

9.4.4 Chaîne 4 (C4) - Client via Assisteur / OF prive (mutuelle)

9.4.5 Chaîne 5 (C5) - Mandataire + avoir (annulation hors delai puis geste commercial)

9.4.6 Chaîne 6 (C6) - Multi-aides + matching client/intervenant

9.4.7 Chaîne 7 (C7) - Franchise + paie intervenant (MP7)

9.4.8 Chaîne 8 (C8) - Clôture mensuelle de facturation

9.4.9 Chaîne 9 (C9) — Cycle complet PP : de l'onboarding à la paie (chaîne maîtresse)

9.5 Matrice de couverture des chaînes

Module / capacité	C1 (PP)	C2 (Pro)	C3 (SAAD)	C4 (Assist.)	C5 (Avoir)	C6 (Multi)	C7 (Franch.)	C8 (Clôture)	C9 (Complet)
MP1 — Structure	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui
MP2 — Catalogue	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui
MP3 — Collaborateurs	Oui	Oui	Oui	Oui	Oui	Oui	Oui	(Opt)	Oui
MP4 — Commercial	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui
MP4 — SEGUR	—	—	Oui	—	—	—	—	—	—
MP4 — Assistant (SP4.5)	—	—	—	Oui	—	—	—	—	—
MP5 — Planning / RRULE	Oui	Oui	Oui	Oui	(Opt)	Oui	Oui	—	Oui
MP5 — Intervention ponctuelle	—	(Opt)	—	—	Oui	—	—	—	—
MP5 — Annulation	Oui	—	—	—	Oui	—	—	—	Oui
MP5 — Absences client	—	Oui	—	—	—	—	—	—	—
MP5 — Matching	—	—	—	—	—	Oui	(Opt)	—	—
MP6 — Aides / PEC	Oui	—	Oui	—	—	Oui	—	—	Oui
MP6 — Collecte facturation	Oui	(Opt)	Oui	Oui	Oui	Oui	—	Oui	Oui
MP6 — Avoir / non-facturation	Oui	—	—	—	Oui	—	—	—	—
MP7 — Paie (export heures)	—	—	—	—	—	—	Oui	—	Oui
Notifications Kafka	Oui	Oui	(Opt)	Oui	Oui	Oui	Oui	Oui	Oui

Légende : Oui = couvert | (Opt) = couvert en option / partiellement | — = non couvert par cette chaîne.

9.6 Jeux de données et gestion des dépendances

- **Autoportance.** Chaque chaîne crée ses propres données (pas de dépendance à un état DB préexistant).
- **Séquencement.** Les étapes s'exécutent dans l'ordre ; les identifiants produits sont capturés et injectés dans les étapes suivantes.
- **Isolation.** Un agency_id dédié par exécution ; préfixe TEST_AUTO_ ; noms fictifs (RGPD) ; SIREN/NIR factices.
- **Nettoyage.** Teardown en fin de chaîne ou restauration d'un snapshot avant/après.
- **Batch et externes.** Batch RRULE déclenché via API ou mocké ; INSi/iCanopée via bac à sable ou virtualisation.10. Environnements de Test

La stratégie cible une chaîne d'environnements couvrant l'ensemble du cycle. L'analyse terrain montre qu'à ce jour trois environnements sont documentés (DEV1, INT1, SQE1) ; les environnements PREPROD et PROD sont à formaliser. Le tableau distingue la cible et l'état actuel.

10. Environnements cible

10.1. Chaîne d'environnements cible

Env.	Utilisation	Jeux de données	Rafraîchissement	Responsabilité	État actuel
DEV	Développement, tests unitaires et intégration locaux	Testcontainers / données synthétiques	À la demande (éphémère)	Développeurs	En place (DEV1)
INT	Intégration continue, tests fonctionnels internes, E2E smoke	Seed API + snapshot baseline	Par sprint / au fil de l'eau	Équipe + QA	En place (INT1)
QA	Campagnes de test dédiées, non-régression, exploratoire	Snapshot maîtrisé, anonymisé	Par campagne	QA transverse	À formaliser
RECETTE	Recette métier / client (UAT)	Données représentatives anonymisées	Par version figée	Métier / PO + QA	En place (SQE1)
PREPROD	Iso-production : perf, sécurité, smoke pré-MEP	Volumétrie proche prod (anonymisée)	Avant chaque MEP	DevOps + QA	À créer (recommandé)
PROD	Production	Données réelles	— (supervision continue)	DevOps / Exploitation	À venir (GA)

10.2 Environnements documentés (constat)

Environnement	URL	Périmètre / usage constaté
DEV 1	https://orion-dev1.itsap.net	CRM, aides financières, planning ; déploiement au fil de l'eau après merge
INT 1	https://orion-int1.itsap.net	Tests internes/fonctionnels ; données séparées (planning MP5)
REC 1 (SQE)	https://orion-sqe1.itsap.net	Recette ITSAP ; par version figée

Infrastructure : région AWS eu-central-1, accès bases via pgAdmin (tunnel SSM/bastion EC2), une instance RDS PostgreSQL par domaine (CRM, Facturation/Paie, Gestion Admin, Gestion Marque, SAP, Offres/Tarifs, Transverse).
Orchestration Nomad.

10.3 Stratégie de gestion des données de test

Stratégie	Quand l'employer	Avantage	Limite
Seed via API	Tests CRUD, chaînes E2E	Données fraîches, test isolé	Nécessite des API disponibles, plus lent
Snapshot DB	Suite complète en CI	Rapide, reproductible	Données figées, dérive si le schéma évolue

Stratégie	Quand l'employer	Avantage	Limite
Données autoportantes	Tests idempotents	Zéro dépendance externe	Cleanup plus complexe
Fixtures statiques	Référentiels, configuration	Simple, versionné	Se désynchronise du modèle réel

Mapping recommandé par couche Orion:

- **Structure (MP1) & Catalogue (MP2).** Snapshot DB (données de référence stables, incluses dans le baseline).
- **Collaborateurs (MP3).** Seed API au démarrage de la suite (globalSetup).
- **Commercial (MP4) & Planning (MP5).** Seed API par test (beforeEach) + nettoyage (afterEach).
- **Facturation (MP6).** Snapshot (configuration) + seed API (plans d'aide).
- **SEGUR (MP4).** Mock iCanopée + données pré-injectées.

Recommandation environnements

Ajouter un environnement PREPROD iso-production pour fiabiliser les tests de performance, de sécurité et les smoke tests pré-MEP, aujourd'hui non couverts.

Formaliser un environnement QA dédié aux campagnes (non-régression, exploratoire) afin de ne pas saturer INT1, partagé avec l'intégration continue.

Industrialiser le rafraîchissement des jeux de données (snapshot anonymisé reproductible) et l'anonymisation systématique (RGPD).

11. Stratégie d'Automatisation

L'automatisation est le socle de la stratégie (Automation First). Elle s'appuie sur un outillage cible et des critères explicites de priorisation, de ROI et de maintenance.

11.1 Outillage d'automatisation

Outil	Rôle dans l'automatisation	Périmètre
Playwright	Automatisation E2E web (TypeScript), multi-navigateurs, interception réseau	Parcours critiques Angular
OpenCode	Assistance IA à la génération/maintenance des tests et à l'analyse des sources (specs, US)	Productivité QA / SDET
Xray	Référentiel de tests, plans, exécutions ; consolidation manuelle + automatisé ; traçabilité	Gestion des tests (Jira)
GitHub Actions	Orchestration CI des suites (build, unit, intég, contrat, E2E)	Pipelines applicatifs
GitLab CI	Pipelines (notamment plateforme Data / GitOps) et déploiements	Infra / Data / déploiement

Outil	Rôle dans l'automatisation	Périmètre
Pact	Contract testing consumer-driven (broker + can-i-deploy)	APIs et Kafka
k6	Tests de performance automatisés (charge, stress, endurance)	Non fonctionnel
OWASP ZAP	Scans de sécurité automatisés (DAST)	Non fonctionnel

11.2 Critères d'automatisation

Un cas de test est candidat à l'automatisation s'il satisfait plusieurs des critères suivants :

- **Répétabilité.** Rejoué à chaque release (régression) ou fréquemment.
- **Criticité.** Couvre un parcours critique ou une règle de gestion sensible.
- **Stabilité.** Fonctionnalité suffisamment stable pour un ROI positif (éviter d'automatiser une UI mouvante).
- **Déterminisme.** Résultat prévisible et vérifiable (ex. RRULE, calculs, contrats).
- **Coûts manuels élevés.** Cas long ou fastidieux à exécuter manuellement.

À l'inverse, restent manuels : l'exploratoire, l'ergonomie/UX subjective, les cas ponctuels à faible ROI et les fonctionnalités instables.

11.3 Priorisation (score de risque)

La priorité d'automatisation combine criticité, fréquence, risque de régression et complexité (score sur 20).

Module	Criticité	Fréquence	Risque régr.	Complexité	Score /20	Priorité
MP5 Planning	5	5	5	5	20	P0
MP1 Structure	5	4	4	4	17	P0
MP6 Facturation	5	3	4	5	17	P1
MP4 Commercial	4	5	3	3	15	P1
MP3 Collaborateurs	4	4	3	3	14	P2
MP7 Paie	4	2	2	3	11	P3
MP2 Catalogue	3	3	2	2	10	P3

11.4 Phases d'automatisation

Phase	Sprints	Périmètre	Types de tests
1 — Fondations	1-3	MP5 Planning + MP1 Habilitations + contrat Kafka	Unit. RRULE, intég. Kafka, intég. Keycloak
2 — Extension	4-6	MP6 Facturation + MP4 Commercial + E2E web	Unit. PEC, intég. SIRENE/SEGUR, 5 E2E
3 — Consolidation	7-9	MP3 Collaborateurs + MP2 Catalogue + E2E mobile	Unit. Annualisation, intég. GTA, E2E mobile
4 — Complétion	10-12	MP7 Paie + performance + sécurité	Intég. paie, k6, OWASP ZAP

11.5 ROI de l'automatisation

Le ROI se mesure par la réduction du coût et du délai de non-régression, et par la valeur des défauts détectés tôt. Une campagne de régression manuelle (plusieurs jours-personne, une fois par release) est remplacée par une exécution automatisée de l'ordre de la dizaine de minutes, rejouable à chaque changement. Le seuil de rentabilité est atteint sur l'horizon du programme (estimation ~12-15 mois), avec un bénéfice croissant à mesure que la suite s'étoffe.

11.6 Maintenance et gestion des tests instables (flaky)

- **Budget flaky.** Maximum 2 % de la suite ; au-delà, un sprint de stabilisation dédié est déclenché.
- **Politique de retry.** Unitaire : 0 retry (un échec = un bug). Intégration : 1 retry, investigation sous 48 h. E2E : 2 retries, mise en quarantaine si non résolu sous 5 jours.
- **Quarantaine.** Un test instable est isolé (@quarantine) pour ne pas bloquer la CI, avec ticket de correction.
- **Anti-patterns proscrits.** Identifiants en dur, dates absolues, dépendance à l'ordre d'exécution, absence de nettoyage, données réelles.
- **Conventions de nommage.** [Module]_[Fonctionnalité]_[Scénario]_[RésultatAttendu].

Niveau	Retry	Tolérance flaky	Action si dépassement
Unitaire	0	0 %	Correction immédiate (= bug)
Intégration	1	< 1 %	Investigation sous 48 h
E2E	2	< 2 %	Quarantaine si non résolu sous 5 j
Performance	0	N/A	Investigation, pas de retry

12. Outillage

L'écosystème d'outils couvre la gestion de projet, la documentation, la gestion des tests, l'automatisation, la qualité du code, la sécurité et l'observabilité. Chaque outil est positionné par usage, responsabilité et intégration.

Outil	Usage	Responsabilité	Intégration
Jira (projet ORI)	Backlog, Epics/US, anomalies, workflow	PO, équipes, QA	Source des exigences ; lien vers Xray et Confluence
Confluence (espace Orion)	Spécifications, règles de gestion, ODR, DoR/DoD	PO, BA, Architecte	Référence documentaire liée aux US
Xray for Jira	Cas de test, plans, exécutions, traçabilité	QA / Test Lead	Consolidation manuelle + CI ; lien US↔Test
GitHub	Code, revues, GitHub Actions (CI)	Développeurs, DevOps	Déclenche pipelines et gates ; pousse résultats Xray
GitLab	Pipelines (Data/GitOps), déploiements	DevOps	CI/CD, déploiement infra
Playwright	Automatisation E2E web	QA / SDET	Intégré à la CI ; rapports ; annotations Xray

Outil	Usage	Responsabilité	Intégration
OpenCode	Assistance IA (génération/maintenance de tests, analyse sources)	QA / SDET	Accélère la production de tests et l'exploitation des specs
SonarQube	Analyse statique, quality gate, couverture	Développeurs, Tech Lead	Gate G4 bloquante en CI
OWASP ZAP	Scan de sécurité dynamique (DAST)	QA sécurité / RSSI	Gate G8 (release), scans nightly
Pact (broker)	Contract testing (API, Kafka)	Équipes + QA transverse	Gate G5 (can-i-deploy)
k6	Tests de performance	QA transverse / DevOps	Nightly + pré-release ; export Grafana
Prometheus	Collecte de métriques (Shift Right)	DevOps / SRE	Supervision applicative et infra
Grafana	Tableaux de bord, alerting	DevOps / QA	Visualisation perf, santé CI, KPI

Recommandation outillage — activer la traçabilité Xray

Constat : Xray est installé mais aucun cas de test n'est relié aux User Stories ORI ; Jira n'utilise ni composants, ni versions (fixVersion), ni labels.

Action : relier chaque US critique à au moins un test Xray ; introduire des fixVersions (V0.1, V0.2, GA) et des labels (@critical, module) pour le pilotage ; pousser automatiquement les résultats CI vers des Test Executions Xray.

13. Gouvernance

La gouvernance qualité définit les rôles, responsabilités et instances de décision. Elle s'appuie sur une matrice RACI détaillée (R = Réalise, A = Approuve/Responsable final, C = Consulté, I = Informé) couvrant les activités clés du cycle de test.

13.1 Rôles et responsabilités

Rôle	Responsabilité principale en matière de qualité
QA Manager	Définit et pilote la stratégie de test ; suit les KPI ; arbitre les priorités qualité ; rend compte à la direction de programme.
Test Lead	Coordonne les activités de test ; gère les plans et campagnes Xray ; anime la QA transverse.
Test Analyst	Conçoit et exécute les cas de test ; analyse les résultats ; qualifie les anomalies.
Dev Lead / Tech Lead	Garantit la testabilité technique ; approuve les tests de contrat et l'architecture de test ; assure les gates techniques.
Développeur	Écrit et maintient les tests unitaires et d'intégration de son périmètre ; corrige les anomalies.
Product Owner (PO)	Définit les critères d'acceptation ; valide la recette métier ; priorise le backlog et les anomalies.
Business Analyst (BA)	Formalise les règles de gestion ; assure la cohérence specs↔tests ; appuie la recette.
Architecte	Définit les ODR ; valide les stratégies d'intégration (Kafka, API, multi-tenant) ; approuve les tests non fonctionnels.
DevOps / SRE	Fournit et maintient environnements et pipelines ; intègre les gates ; opère l'observabilité (Shift Right).
RSSI	Définit les exigences de sécurité ; valide les tests de sécurité et la conformité (OWASP, RGPD, IAM).

13.2 Matrice RACI des activités de test

Activité	QA Mgr	Test Lead	Test Analyst	Dev Lead	Dev	PO	BA	Archi	DevOps	RSSI
Définir la stratégie de test	A	R	C	C	I	C	C	C	C	C
Rédiger la DoR / DoD	C	C	I	C	I	A	R	C	I	I
Écrire les tests unitaires	I	I	C	C	R	I	I	I	I	I
Écrire les tests d'intégration	I	C	C	A	R	I	I	C	C	I
Contract testing (provider)	I	C	C	A	R	I	I	C	C	I
Contract testing (consumer)	I	R	C	A	C	I	I	C	C	I
Tests E2E (web/mobile)	C	A	R	C	C	C	I	I	C	I
Tests de performance	C	A	R	C	C	I	I	C	R	I

Activité	QA Mgr	Test Lead	Test Analyst	Dev Lead	Dev	PO	BA	Archi	DevOps	RSSI
Tests de sécurité	C	C	C	C	C	I	I	C	C	A
Recette métier (UAT)	C	C	C	I	I	A	R	I	I	I
Gestion des anomalies	A	R	R	C	R	C	C	I	C	I
Go / No-go de release	A	C	I	C	I	C	I	C	R	C
Suivi des KPI qualité	A	R	C	I	I	I	I	I	C	I
Maintenance des tests	I	A	R	C	R	I	I	I	C	I

Légende. R = Réalise • A = Approuve (responsable final) • C = Consulté • I = Informé.

13.3 Instances et rituels qualité

- **Revue de stratégie par jalon.** À chaque milestone (V0.1, V0.2, GA), revue et mise à jour de la stratégie.
- **Rétrospective QA mensuelle.** QA + un développeur par module ; analyse des KPI, du flaky, des défauts échappés.
- **Comité go/no-go de release.** Vérification des quality gates et des exit criteria avant mise en production.
- **Veille et montée en compétence.** Veille technologique régulière, ateliers TDD/BDD trimestriels.

14. Critères d'Entrée (Entry Criteria)

Les critères d'entrée définissent les conditions préalables au démarrage de chaque niveau de test. Ils garantissent que l'activité ne débute pas prématurément, évitant le gaspillage et les faux résultats.

Niveau de test	Critères d'entrée
Unitaire	Code compilé ; règle de gestion / spécification disponible ; environnement de build opérationnel.
Intégration	Tests unitaires verts ; conteneurs (Testcontainers) démarrent ; schémas de données et contrats d'interface disponibles ; mocks des dépendances externes prêts.
Contrat	Contrats (Pact / OpenAPI) définis et versionnés ; broker accessible ; producteurs et consommateurs identifiés.
API	Endpoints déployés sur l'environnement cible ; spécification OpenAPI à jour ; jeux de données d'API prêts.
E2E (web/mobile)	Environnement intégré disponible et sain (health check) ; parcours spécifié ; jeux de données de référence préchargées automatiquement dans une base de données afin de rendre l'application immédiatement utilisable ou testable ; DoR satisfaite pour les US couvertes.
Performance	Environnement iso-prod (ou dédié) stable ; SLA cibles définis ; volumétrie de référence disponible ; monitoring actif.
Sécurité	Périmètre et exigences de sécurité définis (RSSI) ; environnement cible autorisé pour le scan ; comptes de test prêts.
Recette métier (UAT)	Fonctionnalités livrées en recette ; critères d'acceptation validés ; anomalies bloquantes précédentes corrigées ; données représentatives disponibles.

Préalable transverse. *Aucune campagne ne démarre sans que la Definition of Ready (DoR) des User Stories concernées soit satisfaite : US validée par le PO, critères d'acceptation définis, maquettes fournies si impact front, dépendances identifiées, US estimée.*

15. Critères de Sortie (Exit Criteria)

Les critères de sortie définissent les conditions objectives d'achèvement de chaque niveau de test. Ils conditionnent la progression dans le pipeline et, in fine, la décision de release.

Niveau de test	Critères de sortie
Unitaire	100 % des tests verts ; couverture ≥ 80 % (bloquant 75 %) ; aucune régression sur les tests existants.
Intégration	Tests verts ; couverture ≥ 70 % (bloquant 60 %) ; flux critiques (Kafka, persistance, auth) validés.
Contrat	100 % des contrats vérifiés (provider + consumer) ; can-i-deploy au vert ; aucune rupture non coordonnée.
API	Tous les endpoints critiques verts ; codes de statut et schémas conformes ; cas d'erreur (401/403/404/5xx) couverts.
E2E (web/mobile)	100 % des parcours critiques verts ; taux de flaky < 2 % ; captures/traces disponibles pour analyse.
Performance	SLA respectés (p95) sur les scénarios cibles ; pas de dégradation vs baseline ; pas de fuite en endurance.
Sécurité	0 vulnérabilité critique ou haute ouverte ; conformité OWASP vérifiée ; isolation multi-tenant démontrée.
Recette métier (UAT)	Critères d'acceptation satisfaits ; 0 anomalie bloquante/critique ouverte ; PV de recette approuvé par le PO.

15.1 Critères de Release Readiness (go/no-go)

Au niveau programme, la mise en production est conditionnée à la satisfaction cumulée des critères suivants :

Critère de release	Seuil requis	Statut requis
Quality gates G1–G8	Toutes au vert	Bloquant
Parcours critiques E2E	100 % verts	Bloquant
Anomalies bloquantes / critiques	0 ouverte	Bloquant
Vulnérabilités critiques	0 ouverte	Bloquant
Couverture unitaire globale	$\geq$ seuil du jalon	Bloquant
Tests de performance (release)	SLA respectés	Bloquant (release majeure)

Critère de release	Seuil requis	Statut requis
Anomalies majeures	Sous seuil convenu, avec plan	Non bloquant (arbitrage QA Mgr/PO)
PV de recette métier	Approuvé	Bloquant

16. Indicateurs Qualité

Les indicateurs qualité (KPI) pilotent la stratégie dans la durée. Chaque indicateur est associé à un objectif, un seuil d'alerte, une fréquence de mesure et un responsable. Ils sont visualisés dans des tableaux de bord (Grafana / Xray) et revus lors des rituels qualité.

KPI	Définition	Objectif	Seuil d'alerte	Fréquence	Responsable
Coverage	Couverture de code par les tests (unit. /intég.)	≥ 80 % / ≥ 70 %	< 75 % / < 60 %	Chaque build	Dev Lead
Defect Density	Défauts rapportés au volume de code livré	< 1 majeur / 1000 LoC	> 2 / 1000 LoC	Par sprint	QA Manager
Defect Leakage	Part des défauts détectés en production vs total	< 5 %	> 10 %	Par release	QA Manager
MTTR	Délai moyen de restauration (test cassé / incident)	< 4 h	> 8 h	Continu	Test Lead / DevOps
MTBF	Temps moyen entre deux défaillances (stabilité)	En hausse continue	Baisse 2 périodes	Par release	DevOps / SRE
Test Pass Rate	Taux de tests réussis sur la suite	≥ 98 %	< 95 %	Chaque build	Test Lead
Flaky Rate	Taux de tests instables	< 2 %	> 5 %	Hebdomadaire	Test Lead
Build Success Rate	Taux de builds CI réussis	≥ 95 %	< 90 %	Continu	DevOps
Release Readiness	Indice de préparation à la release (gates satisfaites)	100 %	< 100 %	Par release	QA Manager

16.1 Codes couleur des seuils (pilotage visuel)

Métrique	Vert	Orange	Rouge
Couverture unitaire	≥ 80 %	60 – 79 %	< 60 %
Couverture intégration	≥ 70 %	50 – 69 %	< 50 %
Taux de réussite	≥ 98 %	95 – 97 %	< 95 %
Taux de flaky	< 1 %	1 – 2 %	> 2 %
Temps' unit. + intég.	< 10 min	10 – 15 min	> 15 min
Temps E2E	< 30 min	30 – 45 min	> 45 min
Vulnérabilités critiques	0	1 – 2	≥ 3

Métrique	Vert	Orange	Rouge
Défauts échappés / release	0	1 – 2	≥ 3

16.2 Alertes automatiques

Déclencheur	Action automatique
Chute de couverture > 5 %	Investigation immédiate, identification de la MR responsable
Temps CI au-delà du seuil	Optimisation du pipeline (parallélisation, sélection)
Flaky > 2 %	Planification d'un sprint de stabilisation
0 test ajouté alors que des features sont livrées	Avertissement en rétrospective
Vulnérabilité critique détectée	Correction sous 24 h, blocage de release
E2E rouge sur 2 exécutions consécutives	Gel des merges (merge freeze) sur develop

17. Roadmap

La mise en œuvre de la stratégie suit une trajectoire progressive, alignée sur les jalons produit (V0.1, V0.2, GA). Elle privilégie les quick wins à fort ROI en début de parcours, puis étend la couverture par vagues.

17.1 Phase 1 — Fondations (quick wins)

- **Objectif.** Établir le socle : automatisation des modules les plus critiques et des interfaces, mise en place des gates et de la traçabilité.
- **Priorités.** MP5 Planning (batch RRULE, annulation), MP1 habilitations/cône, contract testing Kafka, gates G1–G6 en CI, liaison Xray↔US.

Quick win	Effort	Justification
Contract testing Kafka	2–3 j	Schémas définis, format normalisé, outillage prêt
Tests API BFF (sync mobile)	3–4 j	Endpoints documentés, règles claires
Tests d'habilitations	2–3 j	Matrice RBAC existante
Tests unitaires RRULE	2–3 j	RFC 5545 normée, cas déterministes
Tests de validation des formulaires	1–2 j	Règles transverses documentées
Liaison Xray ↔ User Stories	2–3 j	Outillage présent, active la traçabilité

17.2 Phase 2 — Extension

- **Objectif.** Étendre la couverture aux calculs sensibles et aux parcours métier complets.
- **Priorités.** MP6 Facturation (PEC, import Excel), MP4 Commercial (SIRENE, SEGUR), premiers E2E web (5 parcours), baseline de performance, scans OWASP.
- **Gains attendus.** Détection précoce des erreurs de calcul financier ; non-régression des parcours clés ; visibilité sécurité.

17.3 Phase 3 — Consolidation & GA

- **Objectif.** Compléter la pyramide, industrialiser le mobile et le non-fonctionnel, atteindre les cibles GA.
- **Priorités.** MP3 Collaborateurs (annualisation, GTA), MP2 Catalogue, E2E mobile (Hyperion), performance validée, pen-testing, MP7 Paie (dès specs disponibles).
- **Gains attendus.** Suite de non-régression complète ; ROI d'automatisation atteint ; go/no-go outillé.

17.4 Trajectoire de couverture par jalon

Jalon	Échéance	Unit.	Intég.	Contrat	E2E	Perf	Sécurité
V0.1 Recette	T1 2026	≥ 60 %	≥ 50 %	100 %	5 parcours	—	—
V0.2	05/2026	≥ 75 %	≥ 65 %	100 %	10 parcours	Baseline	OWASP
V1.0 GA	08/2026	≥ 80 %	≥ 70 %	100 %	20 parcours	Validé	Pen-test

18. Conventions et Industrialisation

Cette section regroupe les conventions transverses qui garantissent l'homogénéité, la maintenabilité et la traçabilité de l'effort de test à l'échelle du programme.

La CI/CD est organisée en stages avec durées cibles, la virtualisation des dépendances externes est assurée via WireMock et Testcontainers. Une taxonomie de tags homogénéise les tests. Des règles transverses UI et spécificités mobile offline-first sont définies.

18.1 Intégration continue — stages et déclencheurs

Événement	Stages exécutés	Durée cible
Push (feature)	Build + Unitaire	< 5 min
Merge Request → develop	Build + Unitaire + Intégration + Contrat	< 15 min
Merge sur develop	+ E2E smoke	< 20 min
Nightly	Suite complète	< 60 min
Merge Request → release/*	Complet + E2E étendu	< 45 min

18.2 Stratégie de virtualisation des dépendances externes

Dependence externe	Solution	Scenarios simulés
SIRENE (INSEE)	WireMock	200, 404 (inconnu), 429 (rate limit), 503, timeout
GEOWS (géocodage)	WireMock	200, 404 (introuvable), timeout
iCanopée (SEGUR)	WireMock / bac à sable	201, 401 (token expiré), 500
Keycloak	Testcontainers	Auth, rôles, isolation de tenant
Kafka	Testcontainers	Topics dynamiques par test
Redis	Testcontainers	Cache, TTL, invalidation
FCM (Firebase)	WireMock	200, 404 (token invalide), 429 (quota)

18.3 Conventions de nommage et taxonomie de tags

Nommage. [Module]_[Fonctionnalité]_[Scénario]_[RésultatAttendu]. Exemples:

MP5_RRULE_WeeklyRecurrenceWithExdate_SkipsExcludedDates;

MP1_Habilitations_UnauthorizedAccess_Returns403.

Catégorie de tag	Valeurs
Criticité	@smoke, @critical, @regression, @extended
Type	@unit, @integration, @contract, @e2e, @performance, @security
Module	@mp1, @mp2, @mp3, @mp4, @mp5, @mp6, @mp7, @transverse
Plateforme	@web, @mobile, @api, @batch
Technique	@kafka, @keycloak, @bff, @rrule, @offline
État	@quarantine, @wip

18.4 Tests transverses (règles UI communes)

Les règles transverses (pagination, modales, filtres, habilitations, notifications) sont validées une fois et rejouées sur tous les écrans concernés.

ID	Scénario transverse	Résultat attendu
TX-MOD-01	Navigation multi-étapes (Suivant)	Aucun appel API prématuré (interception réseau)
TX-MOD-03	Enregistrement principal	Validation + fermeture modale (Après la réussite de l'enregistrement, la fenêtre modale (popup) se ferme automatiquement) + notification (snackbar).
TX-MOD-07	Fermeture avec modifications non enregistrées	Modale de confirmation Quitter / Enregistrer
TX-PAG-01	Pagination (défaut 15; options 10/15/20/25)	Compteur « X - Y sur Z » ; bornes désactivées

ID	Scénario transverse	Résultat attendu
TX-FIL-01	Filtres conservés dans un domaine	Réinitialisés au changement de micro-service ; retour navigateur restaure
TX-HAB-03	Appel API en contournant l'UI	HTTP 403
TX-HAB-05	Accès à une entité hors cône (IDOR)	HTTP 403 ou 404
TX-HAB-07	JWT modifié (injection de rôle)	Rejet (signature invalide)

18.5 Spécificités mobile (Hyperion, offline-first)

- **Offline-first.** Fonctionnement complet depuis SQLite en mode avion ; reprise de synchronisation au retour réseau, sans action utilisateur, sans corruption.
- **Synchronisation.** Server_ts = max(updated_at) du delta (jamais l'horloge locale) ; tombstones DELETE purgées localement ; lots de 200 en full load.
- **Sécurité mobile.** SQLite chiffré (SQLCipher) illisible sans clé ; JWT en Keychain/Keystore, refresh automatique, rejet si altéré.
- **Performance mobile.** Synchronisation initiale < 10 s (4G) ; delta < 2 s ; cold start < 3 s ; scroll 60 fps ; aucune fuite mémoire sur 100 cycles.

19. Recommandations et Plan d'Action

Cette section synthétise les écarts constatés entre la cible et l'état actuel du programme (analyse Jira / Confluence), et propose un plan d'action priorisé pour combler ces écarts.

Les écarts majeurs sont l'absence de stratégie formalisée, traçabilité Xray non reliée, structuration Jira insuffisante, manque d'environnements PREPROD/PROD, couverture incomplète MP2/MP7, absence de quality gates CI, gestion de données à industrialiser. Le plan d'action priorise la diffusion de la stratégie, mise en place des gates, liaison Xray-US, Automatiser MP5 (RRULE, annulation) et contrat Kafka, création d'environnement PREPROD, anonymisation, complétion specs et tests, ajout gates performance/sécurité, inventaire micro-services.

19.1 Écarts constatés vs cible

Domaine	Constat (état actuel)	Cible	Criticité
Stratégie de test	Absente de Confluence	Référentiel formalisé et diffusé	Haute
Traçabilité Xray	Xray non relié aux US	US critiques ↔ tests tracés	Haute
Structuration Jira	Ni composants, ni versions, ni labels	fixVersions + labels + composants	Moyenne
Environnements	3 environnements (pas de PREPROD/PROD)	Chaîne complète + iso-prod	Haute
Modules non couverts	MP2 et MP7 peu/pas documentés	Specs + tests avant GA	Moyenne–Haute

Domaine	Constat (état actuel)	Cible	Criticité
Catalogue micro-services	Page Confluence vide	Inventaire des services à tester	Moyenne
Quality gates CI	Absentes	Gates G1–G8 bloquantes	Haute
Gestion des données	Pas de snapshot anonymisé industrialisé	Snapshot reproductible + anonymisation	Moyenne

19.2 Plan d'action priorisé

N°	Action	Priorité	Responsable	Horizon
A1	Diffuser et faire adopter la présente stratégie de test	P0	QA Manager	Immédiat
A2	Mettre en place les quality gates G1–G6 dans la CI	P0	DevOps + Dev Lead	Phase 1
A3	Relier les US critiques à des cas de test Xray	P0	Test Lead	Phase 1
A4	Introduire fixVersions et labels dans Jira	P1	PO + QA Manager	Phase 1
A5	Automatiser MP5 (RRULE, annulation) et contrat Kafka	P0	Équipe MP5 + QA	Phase 1
A6	Créer un environnement PREPROD iso-production	P1	DevOps	Phase 2
A7	Industrialiser le snapshot anonymisé des données	P1	DevOps + QA	Phase 2
A8	Compléter les specs MP2 / MP7 puis les tests	P2	BA + équipes	Phase 2–3
A9	Ajouter gates performance (G7) et sécurité (G8)	P1	QA + RSSI	Phase 2–3
A10	Renseigner le catalogue des micro-services	P2	Architecte	Phase 2

Facteurs clés de succès

Ownership partagé : les développeurs possèdent leurs tests unitaires/intégration ; la QA transverse orchestre E2E, non fonctionnel et KPI.

Discipline des gates : une gate rouge bloque — sans exception — pour préserver la crédibilité du dispositif.

Amélioration continue : les KPI et les défauts échappés alimentent une boucle de retour à chaque rétrospective.

20. Glossaire et Référence

20.1 Glossaire

Ce glossaire définit les termes clés utilisés dans la stratégie (QE, Shift Left/Right, Quality Gate, RBT, DoR/DoD, CDC/Pact, DLQ, RRULE, PEC, OF/CDA/PAP, GTA, cône de visibilité, multi-tenant, flaky test, MTTR/MTBF, BFF, SEGUR, INS). Les références documentaires et standards (Jira, Confluence, Xray, ISTQB, OWASP, RGAA, RFC 5545) sont listées pour assurer la cohérence et la conformité de la démarche.

Terme	Définition
QE	Quality Engineering — ingénierie de la qualité intégrée tout au long du cycle.
Shift Left / Right	Anticipation du test (gauche) / observation en production (droite).
Quality Gate	Point de contrôle automatisé et bloquant, aligné sur la DoD.
RBT	Risk Based Testing — priorisation par le risque (probabilité × impact).
DoR / DoD	Definition of Ready / Definition of Done.
CDC / Pact	Consumer-Driven Contracts — tests de contrat pilotés par le consommateur.
DLQ	Dead-Letter Queue — file des messages non traitables (Kafka).
RRULE	Règle de récurrence iCalendar (RFC 5545) pour les séries d'interventions.
PEC	Prise En Charge — aide financière appliquée au client (MP6).
OF / CDA / PAP	Organisme Financier / Convention / Plan d'Aide Personnalisé (MP6).
GTA	Gestion des Temps et des Activités (MP3).
Cône de visibilité	Périmètre d'agences/sociétés accessible à un utilisateur.
Multi-tenant	Isolation stricte des données entre marques (realms Keycloak).
Flaky test	Test au résultat non déterministe (passe/échoue sans changement).
MTTR / MTBF	Temps moyen de restauration / entre défaillances.
BFF	Backend-for-Frontend — couche de synchronisation mobile.
SEGU	Programme de e-santé (INS, DUI, DMP, MSSanté) via iCanopée.
INS	Identité Nationale de Santé (statuts : Récupérée, Qualifiée...).

20.2 Références documentaires

Source	Référence	Usage
Jira	Projet ORI (Epics, User Stories, anomalies)	Périmètre, criticité, traçabilité
Confluence	Espace Orion (specs, règles de gestion, ODR)	Règles métier, décisions d'architecture
Confluence	DOR / DOD (page 340459526)	Critères d'entrée/sortie développement
Confluence	Environnements Orion (page 829227030)	URLs et périmètre des environnements

Source	Référence	Usage
Confluence	ODR-2 (erreurs API), ODR-11 (Kafka), ODR-14 (auth)	Standards techniques à tester
Confluence	MP6 SP6.2 — Architecture technique (851738635)	Flux facturation, DMN, table de faits
Xray	Référentiel de tests (projets ORI / HP)	Gestion des tests et exécutions
Interne	Scénarios transverses E2E + stratégie de données	Chaînes E2E et fixtures

20.3 Standards de référence

- **ISTQB.** Foundation et Advanced Test Management (niveaux de test, critères, gestion des risques).
- **Quality Engineering / DevOps Enterprise.** Continuous Testing, Shift Left/Right, pyramide de test.
- **OWASP.** Top 10 et pratiques de test de sécurité applicative.
- **RGAA / WCAG 2.1 AA.** Accessibilité numérique.
- **RFC 5545 (iCalendar).** Récurrence RRULE du planning.

21. Conclusion

Cette stratégie de test d'entreprise répond à la question fondatrice du programme : « **comment allons-nous assurer la qualité du produit Orion ?** ». La réponse est une démarche de Quality Engineering, pilotée par le risque, industrialisée par l'automatisation et gouvernée par des quality gates objectives, du code jusqu'à la production.

Elle s'appuie sur une analyse factuelle du programme (backlog Jira ORI, documentation Confluence, architecture technique réelle) et assume les écarts constatés en proposant un plan d'action priorisé. Sa valeur réside dans son exécution : adoption par les équipes, discipline des gates, et amélioration continue nourrie par les indicateurs.

Document vivant, cette stratégie sera revue à chaque jalon majeur (V0.1, V0.2, GA) pour rester alignée avec l'évolution du produit, des risques et de la maturité des équipes.

« La qualité n'est jamais un accident ; elle est toujours le résultat d'un effort intelligent. »